

Analysis and Design Document: pPIM Compiler

By Sujith S 22BRS1336, Anirudh Sridhar 22BCE5252 and Logamithran M.R. 22BRS1373

1. Problem Statement

The goal is to design a compiler that translates C++ code containing matrix operations into a custom Instruction Set Architecture (ISA) for a Processing-in-Memory (pPIM) architecture. The ISA is based on the research paper [“Flexible Instruction Set Architecture for Programmable Look-up Table based Processing-in-Memory”](#), which emphasizes: - **LUT-based computation** (reprogrammable cores) - **SIMD parallelism** across DRAM clusters - **Minimal data movement** between memory and processing units

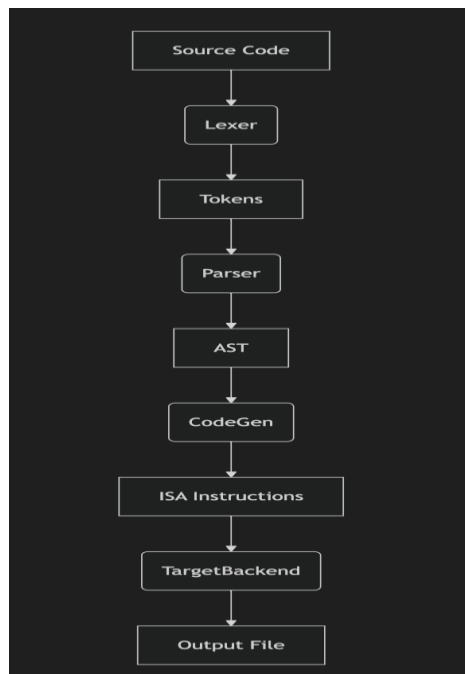
Key Challenges: - Mapping high-level matrix operations (e.g., $C = A * B$) to pPIM's LUT-based microcode sequences. - Managing memory allocation in DRAM subarrays. - Generating ISA instructions for parallel execution across clusters.

2. Design Overview

The compiler follows a traditional pipeline with custom optimizations for pPIM:

C++ Code → Lexer → Parser → AST → CodeGen → ISA → TargetBackend

Control Flow



Key Components

Component	Responsibility
Lexer	Tokenizes input code, detects matrix declarations, and extracts matrix sizes.
Parser	Builds an Abstract Syntax Tree (AST) focused on matrix operations and functions.
CodeGen	Generates pPIM ISA instructions from the AST (PROG/EXE/END commands).
TargetBackend	Validates ISA and writes to output file.

Core Data Structures

Token +type: TokenType +value: string +line: int	ASTNode +type: ASTNodeType +value: string +children: ASTNode[] +line: int	MatrixMap +name: string → address: string	ISAInstruction +opcode: string +operands: string[]
--	---	---	--

3. Detailed Component Analysis

3.1 Lexer (Lexer.h/cpp)

- **Tokenization:**
 - Identifies preprocessor directives (e.g., #define N 256) to extract matrix dimensions.
 - Classifies tokens into matrix declarations (MATRIX_DECL), operators (*, +=), and types (MATRIX_TYPE).
- **Output:** A stream of annotated tokens with line numbers.

3.2 Parser (Parser.h/cpp)

- **AST Construction:**
 - Focuses on functions containing triply nested loops (typical of matrix multiplication).
 - Extracts matrix operands from assignment operations (e.g., C[i][j] += A[i][k] * B[k][j]).
- **Key Data Structures:**
- ```
struct ASTNode {
 ASTNodeType type; // FUNCTION_NODE, MATRIX_OP_NODE, etc.
 std::string value; // "multiply", "*", etc.
 std::vector<std::unique_ptr<ASTNode>> children;
};
```
-

- **Algorithm:**  
**for each** function:  
**for each** statement:  
**if** nested loops **found**:  
**if** += **or** = operation:  
**if** matrix\_access\_pattern:  
     **create** MATRIX\_OP\_NODE

### 3.3 Code Generator (CodeGen.h/cpp)

- **Core Logic:**
  - **Matrix Allocation:** Assigns DRAM addresses to matrices (e.g.,  $A \rightarrow 0x1000$ ).
  - **LUT Programming:** Generates PROG instructions to configure LUT cores for MAC operations.
  - **Operation Mapping:** Translates matrix multiplications into EXE microcode sequences.
- **Key Optimization:** Reuses pre-programmed LUT configurations across operations.
- **Algorithm:**  
**for each** MATRIX\_OP\_NODE:  
     **allocate** matrices  
     **generate** load/store instructions  
     **for** i in 0..matrix\_size:  
         **for** j in 0..matrix\_size:  
             **if** multiplication:  
                 **for** k in 0..matrix\_size:  
                     **generate** MAC operations  
             **if** addition:  
                 **generate** vector add

#### Example ISA Output:

```
MATRIX MULTIPLICATION A * B -> C
PROG r0, mac_operation
EXE MUL r1, ah, bh
EXE MUL r2, al, bl
...
EXE WRITE C_addr[i][j], acc
END matrix_multiply
```

### 3.4 Target Backend (TargetBackend.h/cpp)

- **Validation:** Ensures balanced EXE/END pairs.
  - **Emission:** Writes ISA to a file with debug annotations.
-

## 4. ISA Mapping Strategy

### 4.1 Instruction Types (From Research Paper)

| Instruction | Purpose                                                                 |
|-------------|-------------------------------------------------------------------------|
| PROG        | Programs a LUT core with microcode (e.g., MAC operation).               |
| EXE         | Executes a sequence of micro-operations (multiplication, accumulation). |
| ALLOCATE    | Reserves DRAM rows for matrices.                                        |
| FREE        | Releases memory after computation.                                      |

### 4.2 Matrix Operations → ISA

For a matrix multiplication  $C = A * B$ : 1. **LUT Configuration** (PROG): - Program cores to perform 4-bit multiplies and adds. 2. **Execution** (EXE): - Split 8-bit operands into 4-bit segments. - Route partial products through the cluster router. - Accumulate results in DRAM.

---

## 5. Design Considerations

### 5.1 Memory Management

- **Allocation:** Matrices are allocated contiguously starting at 0x1000.
- **Layout:** Matrices are stored in row-major format to match pPIM's DRAM access patterns.

### 5.2 Parallelism

- **Cluster-Level SIMD:** Identical operations are batched across clusters.
- **Pipeline:** Overlap memory accesses with computations using READ/WRITE queues.

### 5.3 Limitations

- **Control Flow:** Only supports static matrix operations (no branches/loops in ISA).
  - **Precision:** Limited to 8-bit/4-bit fixed-point (per pPIM paper specifications).
- 

## 6. Build System

### CMake Configuration

```
cmake_minimum_required(VERSION 3.20)
project(pPIM_Compiler)
```

```
set(CMAKE_CXX_STANDARD 17)
```

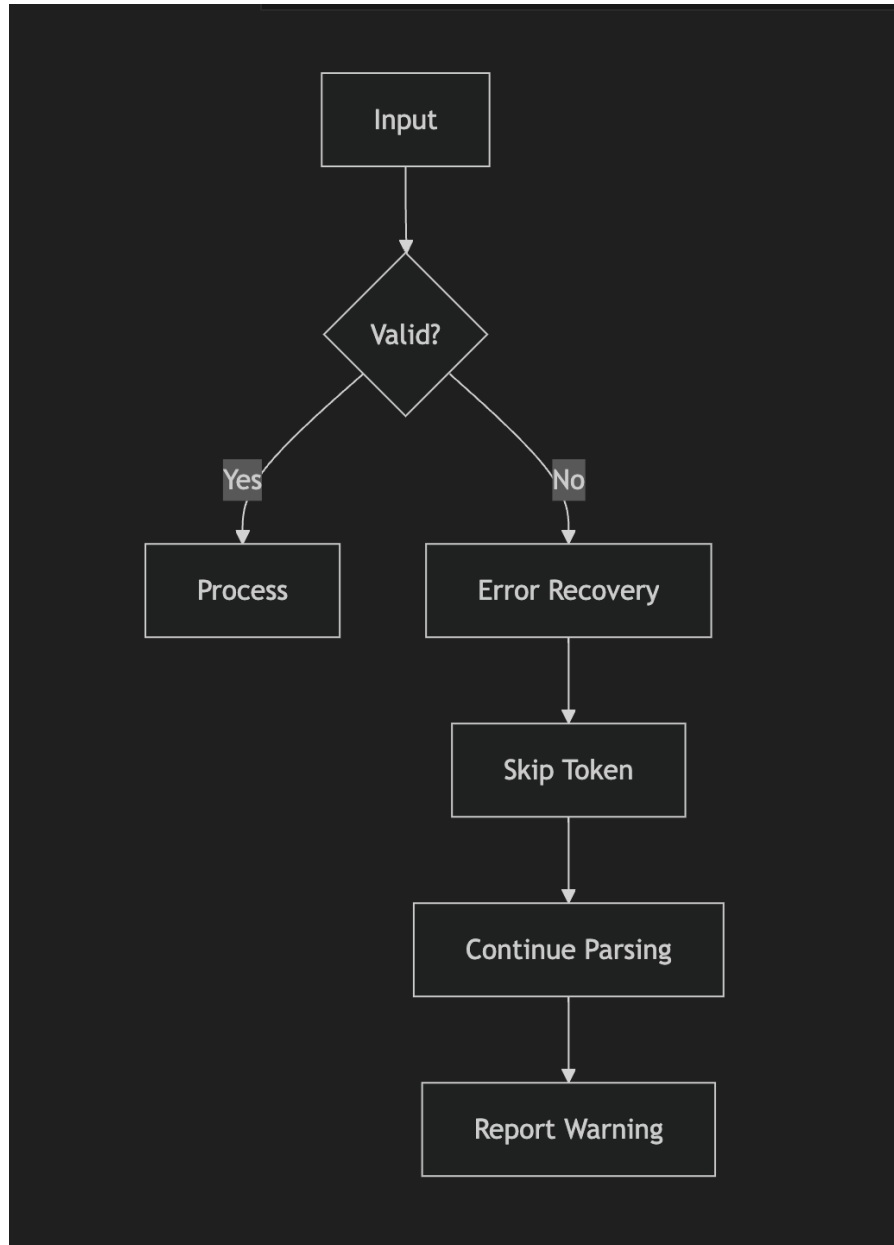
```
add_executable(pPIM_Compiler
 main.cpp
 Lexer.cpp
 Parser.cpp)
```

CodeGen.cpp  
TargetBackend.cpp

)

---

## 7. Error Handling Design



## 8. Performance Characteristics

| Operation         | Time Complexity   | Space Complexity |
|-------------------|-------------------|------------------|
| Lexing            | $O(n)$            | $O(m)$           |
| Parsing           | $O(n)$            | $O(d)$           |
| Matrix Mult Gen   | $O(k^3)$          | $O(k^2)$         |
| Matrix Add Gen    | $O(k^2)$          | $O(k^2)$         |
| Memory Allocation | $O(1)$ per matrix | $O(1)$           |

## 9. Future Work

### 1. Optimization:

- Add peephole optimizations for redundant PROG instructions.
- Implement tiling for large matrices.
- Row-major ordering for efficient caching
- Blocking for large matrices

### 2. Error Handling:

- Add semantic checks for matrix dimension compatibility.

### 3. Extensions:

- Support additional operations (e.g., ReLU, matrix transposition).

---

This design directly addresses the challenges outlined in the research paper, leveraging the pPIM architecture's strengths (LUT-based parallelism, minimal data movement) while adhering to the constraints of the provided ISA.